

ForgeScore Report

legacy.zip

August 14, 2026

70

/100

Established

10 findings (0 critical, 1 high)

Score Breakdown

Dimension	Score	Maturity	Findings
Data Weight	74	Established	2
Code Excellence	73	Established	2
Cognitive Load	76	Advanced	2
System Gravity	78	Advanced	2
Future-Proofing	58	Developing	4
Logic Narrative	70	Established	2
Semantic Clarity	75	Advanced	2
Trust Boundaries	56	Developing	3

Top Strengths

- Boundary validation is centralized: `cleanText`, `readAmount`, `parseWhen`, and `Handler.bodyJson` create a clear validation seam before approval logic receives data.
- Error responses follow a shared path: `Handler.sendProblem` and `Handler.sendJson` reduce ad-hoc response handling and keep invalid input behavior predictable.
- No detected layer violations: The deterministic analysis found 0 layer violations, which keeps trust propagation easier to reason about.
- Domain language is visible: Names like `approve_refund`, `listAudit`, `refund_audit`, `actor`, `action`, and `refund_id` align well with the settlement domain.
- Comments explain why, not just what: The dependency-free Python note and sqlite timeout comment preserve operational context that would otherwise become tribal knowledge.

Findings

High (1)

Finding	Description
Opportunity to make approval authority explicit	<code>approve_refund</code> appears reachable through <code>Handler.do_POST</code> , but the analysis detected no authentication or authorization patterns. A single gateway check would turn approval from an implicit trust path into a coherent handshake.

Medium (8)

Finding	Description
Opportunity to reduce single-file context switching	<code>legacy/server.py</code> mixes HTTP parsing, sqlite lifecycle, validation, audit schema, and business operations. Extracting cohesive modules would lower the mental model count for focused changes.

Finding	Description
Opportunity to deflate the single-file planet	With only legacy/server.py, every change touches the same gravitational center even though the call graph has no cycles. Splitting by concern would reduce change amplification without changing behavior.
Opportunity to add tests as living specification	No test files are present in the supplied tree, so refund approval behavior is not executable documentation. Tests would preserve legacy behavior while making changes safer and faster.
Opportunity to make database lifecycle explicit	getDb silently opens an in-memory sqlite database when _db is unset. That magic path is convenient but can hide misconfiguration and make persistence expectations implicit.
Opportunity to turn the file into a table of contents	legacy/server.py contains transport, validation, database setup, and approval behavior in one source file. Small modules would let the folder structure narrate the system before a developer reads implementation details.
Opportunity to make payload contracts explicit	The analysis found 0 data models, so request and response shapes are implied by Handler.bodyJson and downstream validation. Explicit contracts would reduce assumption-driven bugs when approval fields evolve.
Opportunity to narrow the default network perimeter	DEFAULT_BIND is 0.0.0.0, which makes the legacy approval service listen on all interfaces unless overridden. Safer defaults reduce deployment-time trust assumptions.
Opportunity to document and enforce transport security	The service uses BaseHTTPRequestHandler and HTTPServer directly, so TLS termination is not visible in the code. Making the expected proxy or TLS boundary explicit prevents sensitive approval traffic from depending on tribal deployment knowledge.

Low (10)

Finding	Description
Opportunity to lighten JSON over the wire	The raw HTTPServer path does not show gzip or brotli support for JSON responses. Compression becomes increasingly valuable if listAudit returns larger refund histories.
Opportunity to keep audit reads projection-shaped	The refund_audit table has a small schema today, but listAudit should make its selected fields and limits explicit to avoid future over-fetching. A bounded projection keeps audit payloads light as columns are added.
Opportunity to promote validation as a shared kernel	BadRequest, cleanText, readAmount, and parseWhen form a small internal validation language. Extracting them would reduce repeated boundary code in sibling services.
Opportunity to make the gems easier to compose	The best patterns are currently embedded beside HTTP and sqlite concerns. Separating them would let future handlers snap together validation, response, and audit pieces cleanly.
Opportunity to make configuration discoverable	VERSION, AUTH_LIMIT, DEFAULT_BIND, DEFAULT_PORT, and MAX_BODY are constants in the service file. A small config object would make operational knobs easier to find and validate.
Opportunity to keep Handler as a thin gateway	Handler.sendJson has fan-in 3 and Handler.sendProblem has fan-in 2, while Handler.do_GET and Handler.do_POST orchestrate deeper flows. Keeping Handler thin will prevent transport code from becoming an accidental domain hub.
Opportunity to strengthen type-shaped intent	The service supports old Python and therefore relies on runtime validation rather than typed domain objects. Lightweight dataclasses or named tuples in modern mode would make future changes less telepathic.
Opportunity to document operational constants	DEFAULT_BIND, DEFAULT_PORT, MAX_BODY, and AUTH_LIMIT encode important operational policy. Making their intent explicit reduces breakage when the service is moved or extended.
Opportunity to preserve runtime compatibility decisions	The code clearly supports both Python 2 and newer Python, but the long-term intent is only implied by comments. A short ADR would clarify when this compatibility can or cannot be changed.
Opportunity to name policy constants by intent	AUTH_LIMIT is present, but its exact policy meaning is not visible from the available context. A more precise name would prevent semantic drift around approval thresholds.